

Mobile UI Test Automation

Session 1 — Introduction & Setup

AI-Assisted Mobile Test Automation using Claude

4 Sundays · 2–6 PM SGT

Today: leave with a **green first Appium test** running on an emulator.



 **Let's break the ice**

Drop in the chat:

 **Which city are you joining from?**

Scan the QR code to drop your answer — we'll read a few out while everyone settles in.

Your instructor



Syam Sasi

Workshop Instructor — AI-Assisted Mobile UI
Test Automation

Co-founder of Taqelah 🚀

 linkedin.com/in/syam-sasi

 taqelah.sg

Who this bootcamp is for

- You write (or want to write) **automated UI tests for mobile apps**
- You've used a terminal and read code
- You finished `prerequisites/README.md` — tools installed 

By the end of 4 Sundays you'll: build a real mobile test suite, run it on Android (and iOS), and use **Claude** to write and maintain tests faster.



The 4-Sunday arc

Day	Theme
1 — today	Landscape, setup, first script + locators, core commands, a login flow
2	Writing real, maintainable tests — Page Objects, sync/waits, gestures, data-driven
3	Scaling — reporting, CI, parallel & cloud devices, debugging flaky tests + AI-assisted authoring with Claude begins 🤖
4	More Claude-driven authoring + putting it all together

Each day builds on a **working environment** — so today we make sure yours works.

Today's goal (keep it in mind)

|  Everyone runs a passing Appium test against an emulator before you leave.

Everything else today — the landscape, the architecture — exists to make that test **make sense**, not feel like magic.

A teaser: where Claude comes in

This is an **AI-assisted** bootcamp. Later you'll use:

- **Claude Code** — describe a test in English, get a runnable script
- **taqwright** — a higher-level mobile test runner Claude drives
- **mobile-mcp** — lets Claude *see and tap* the device directly
- **appium-mcp** — lets Claude drive Appium sessions directly

Today we learn the **fundamentals by hand** first. You can't direct an AI tester well if you don't know what it's doing.  → 



The Mobile Testing Landscape

0:10 → 0:35

The two platforms

Android

- OS: Android (many vendors)
- Lang: Kotlin / Java
- Build: **APK / AAB**
- Install: just `adb install` — open
- Devices: huge variety
- Automation engine: **UiAutomator2** / Espresso

iOS

- OS: iOS (Apple only)
- Lang: Swift / Objective-C
- Build: **.app / .ipa**
- Install: needs **code signing**
- Devices: few, controlled
- Automation engine: **XCUITest**

One Appium API drives **both** — that's the whole pitch.



Android builds: APK vs AAB

- **APK** = *Android Package* — the **installable** app file. Devices run this; `adb install app.apk`.
- **AAB** = *Android App Bundle* — a **publishing** format you upload to Google Play.

	APK	AAB
Installs on a device?	✅ directly	❌ Play repackages it first
Purpose	Run / test the app	Ship to the store
Size on device	One-size-fits-all	Play delivers a slimmer, device-specific APK

For Appium we use the `.apk`. (An AAB can be converted with `bundletool`.)

Android engines: UiAutomator2 vs Espresso

	UiAutomator2	Espresso
Style	Black-box — drives device from outside	White-box — runs <i>inside</i> the app
Scope	Any app + the OS (settings, notifications)	Your own app only
Source needed?	No	Yes (wired into the app build)
Speed	Flexible, slightly slower	Faster, auto-syncs with UI thread

- **UiAutomator2** → Appium's **default**; black-box testing the shipped app (*what we use*).
- **Espresso** → devs testing *their own* app in-codebase with tight timing sync.



iOS builds: .app vs .ipa

- **.app** = application **bundle** — the raw build Xcode produces.
- **.ipa** = *iOS App Store Package* — a **zipped, signed** `.app` for real devices.

	.app	.ipa
Runs on	Simulator (no signing)	Real device (needs code signing)
Form	A folder/bundle	A zip wrapping the <code>.app</code>
Use in Appium	Simulator testing	Physical iPhone/iPad

| An `.ipa` is basically a signed, zipped `.app` ready for real devices & distribution.

Listing devices & their status

`adb devices` only shows **running** emulators (and **offline** while booting). To see **every AVD** with a shutdown/running state:

```
# all defined AVDs (any state)
emulator -list-avds
```

```
# running emulators only
adb devices
```

```
Android (adb + emulator):
Pixel_10_Pro_XL      (shutdown)
Pixel_10_Pro_XL_2    (shutdown)
Pixel_10_Pro_XL_3    (shutdown)
taqwright_api34      (shutdown)
```

Boot one → `emulator -avd Pixel_10_Pro_XL` → it appears in `adb devices`.

The friendly **avd: + (status)** listing is what **mobile-mcp / taqwright** prints via `mobile_list_available_devices`.

Installing the app

Android — APK on emulator

```
# states: device / offline / unauthorized
adb devices

# install / reinstall
adb install -r app.apk
adb -s emulator-5554 install -r app.apk
```

AAB? convert with `bundletool` first.

iOS — .app on simulator

```
# states: Booted / Shutdown
xcrun simctl list devices
xcrun simctl boot "iPhone 15"
open -a Simulator

# install + launch
xcrun simctl install booted MyApp.app
xcrun simctl launch booted com.example.MyApp
```

Real device → install signed `.ipa`.

Rule: `.apk` → `adb install` · `.app` → `simctl install` · `.ipa` → real device (signed).

Why the platform differences matter to *testers*

- **iOS needs a Mac.** Simulators + Xcode are macOS-only. Android works anywhere.
- **iOS signing** is a real hurdle (certs, provisioning). Android "just installs."
- **Different element trees & attributes** → same app, different locators per platform.
- So in this course: **Android is the required path**; iOS is an *optional, macOS-only* track.



Real device vs emulator / simulator

	Emulator / Simulator	Real device
Cost	Free	Buy / rent farm
Speed to start	Fast, scriptable	Plug in / cloud
Fidelity	Good enough to learn	Real GPU, sensors, network
When	Dev + most tests	Pre-release, perf, weird bugs

This bootcamp uses emulators/simulators. Same Appium code runs on real devices later — you just change the *capabilities*.

Three kinds of mobile app

Native

Built with the platform SDK.
Renders **real OS widgets**.

Hybrid

Native shell + **a web view**
(HTML/JS inside the app).

Web (mobile web)

Just a **website** in the
phone's browser.

This distinction decides **how you automate it** — so let's make it concrete.

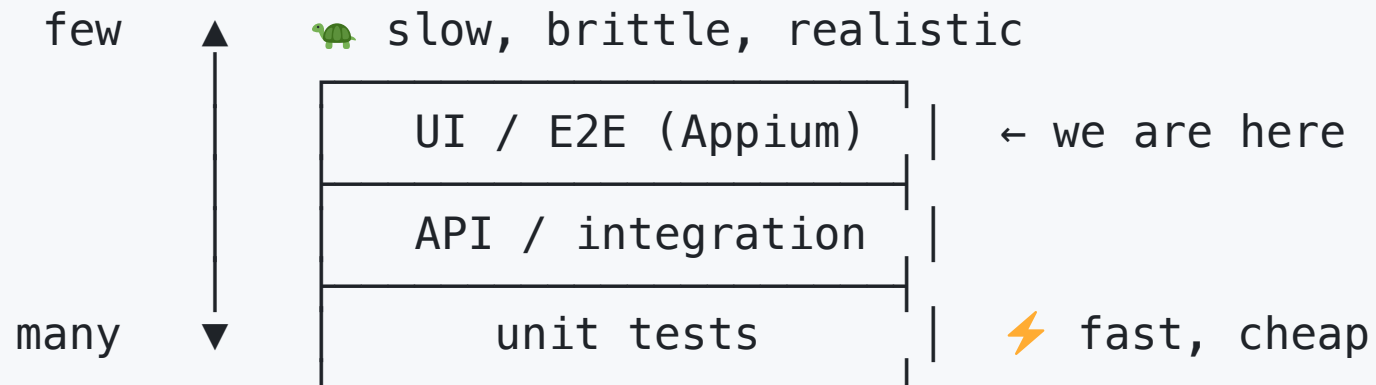
Native vs Hybrid vs Web — how you automate each

	Native	Hybrid	Mobile Web
Made of	OS UI widgets	Native + WebView	HTML in browser
You inspect	UiAutomator2 / XCUITest tree	switch to WEBVIEW context	browser DOM
Locators	id / accessibility id / xpath	CSS/DOM <i>inside</i> webview	CSS / DOM
Appium handles it	✅ natively	✅ via contexts	✅ as a Selenium-style session

Appium calls the native part `NATIVE_APP` and each web view a `WEBVIEW_*` **context**.

You can `getContexts()` and switch between them.

Where UI automation fits



UI tests are **valuable but expensive** — use them for real user journeys, not for logic a unit test could cover.

⚙️ Why Appium? Architecture



0:35 → 1:00

What is Appium?

Appium is WebDriver for mobile.

- An **open-source server** that automates native, hybrid & web apps
- Speaks the **W3C WebDriver protocol** — the same standard Selenium uses
- **Cross-platform**: one API → Android *and* iOS
- **Any language**: Java, Python, JS, Ruby, C#... (it's just HTTP)
- **No app changes required** — you automate the real, shipped app

What is "W3C WebDriver"?

- **W3C** = *World Wide Web Consortium* — the body that sets web standards (HTML, CSS, web APIs).
- **WebDriver** = a **W3C standard** for remotely controlling a browser/app over **HTTP + JSON**.
 - Defines commands like `findElement`, `click`, `sendKeys`, `navigate` ...
 - Vendor-neutral, so *any* client can talk to *any* compliant server.

your test —W3C WebDriver (HTTP + JSON)—> Appium server —> device

- **Selenium** uses it to drive **browsers** · **Appium** uses the *same* protocol to drive **mobile apps**.

Because the protocol is a shared standard, the commands look the same across languages and platforms.



A short history of Appium

When	Milestone
2011	Dan Cuellar (Zoosk) needs faster iOS test automation
2012	Builds <i>iOSAuto</i> on Apple's UIAutomation; Dan presents it as a lightning talk at SeleniumConf 2012 (London) , demoing Selenium-style iOS tests to Jason Huggins. Source goes on GitHub; HTTP server + WebDriver protocol added
Nov 2012	Renamed " Appium " at the Mobile Testing Summit
2013	Sauce Labs backs it; rebuilt on Node.js (Jonathan Lipps). Android + Selendroid support → <i>first truly cross-platform</i> framework
May 2014	Appium 1.0 — becomes the most popular OSS mobile automation framework
2016	Donated to the JS Foundation for long-term OSS stewardship
2023	Appium 2.0 — drivers & plugins ecosystem architecture
2025	Appium 3.0 — modernization, deprecated code removed (<i>what we use today</i>)

The one idea that makes Appium "click"

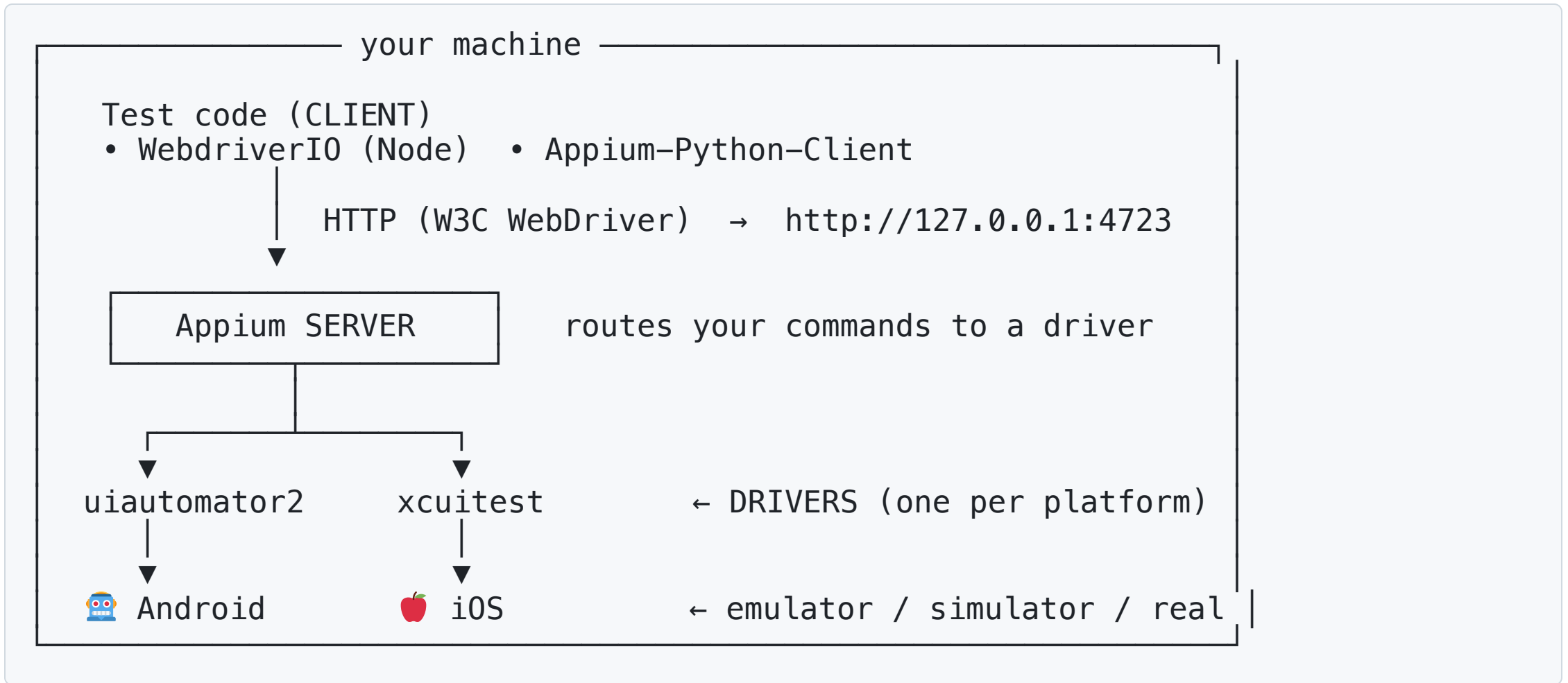
You don't talk to the phone.

You send **HTTP requests** to a **server**, and the server talks to the phone.

```
your test code (any language) —HTTP→ Appium server (port 4723) → driver (per-platform) → device
```

Your test is just a **client**. Swap the language, swap the device — the protocol stays the same.

Architecture — the full picture



Desired capabilities — the "order form"

When your test starts a session, it sends a JSON **capabilities** object that tells the server *what* to automate:

```
{
  "platformName": "Android",
  "appium:automationName": "UiAutomator2",
  "appium:deviceName": "Pixel_10_Pro_XL",
  "appium:appPackage": "com.android.settings",
  "appium:appActivity": ".Settings"
}
```

Change these → run the **same test** on iOS, a real device, or a different app.

We'll edit these live in the lab.

Appium 2 / 3 — the driver model

- Old Appium 1 bundled every driver. **Appium 2+ is a small core + installable drivers.**
- You install only what you need:

```
appium driver install uiautomator2    # Android
appium driver install xcuitest        # iOS (macOS)
appium driver list --installed
```

- We use **Appium 3.x + uiautomator2** (you installed these in prerequisites).



Why Appium (vs the alternatives)?

Tool	Scope	Trade-off
Appium	iOS + Android, any language, native/hybrid/web	One API for everything
Espresso / XCUITest	One platform each (native)	Fast, but platform-locked + in-app
Detox	RN, gray-box	Great for RN only
Maestro	iOS + Android, YAML	Simple, but less programmable

We pick Appium because it's the **industry-standard, cross-platform, language-agnostic** baseline —
and what Claude tooling builds on.

Environment Recap + Verify

1:10 → 1:30

You already installed everything

Full guide: [prerequisites/README.md](#) . Quick recap of the stack:

Tool	Role
Node 26+ + npm	WebdriverIO client + Appium server
Python 3.10+	Appium-Python-Client client
JDK 17	Android automation needs Java
Android Studio + AVD	the device we automate
Appium 3 + uiautomator2	the automation server + driver
Appium Inspector	inspect elements visually

Verify it — run these now (everyone)

```
node -v                # v26+
python3 --version      # 3.10+   (Windows: python --version)
java -version          # 17.x
appium -v              # 3.x
adb devices            # your emulator, "device" not offline
appium driver doctor uiautomator2 # green across the board
```

sos Anything red? Flag it now — see Troubleshooting in [prerequisites/README.md](#). Pair up if stuck.

Appium capabilities — the major ones

Capability	Scope	What it does
<code>platformName</code>	both	OS family — <code>Android</code> / <code>iOS</code> (<i>W3C standard — no prefix</i>)
<code>appium:automationName</code>	both	the driver : <code>UiAutomator2</code> () / <code>XCUITest</code> ()
<code>appium:deviceName</code>	both	device label, e.g. <code>Pixel_10_Pro_XL</code>
<code>appium:udid</code>	both	target a specific device when several are connected
<code>appium:app</code>	both	path/URL to <code>.apk</code> / <code>.ipa</code> / <code>.app</code> → install + launch
<code>appium:appPackage</code>		Android app package id (<code>com.taqelah.demo_app</code>)
<code>appium:appActivity</code>		Android activity to launch (<code>.MainActivity</code>)
<code>appium:autoGrantPermissions</code>		auto-accept runtime permission dialogs
<code>appium:bundleId</code>		iOS app bundle id
<code>appium:noReset</code>	both	keep app data/state between sessions
<code>appium:fullReset</code>	both	uninstall + reinstall for a clean slate
<code>appium:newCommandTimeout</code>	both	idle seconds before the server ends the session

Boot your emulator

```
emulator -list-avds          # see your AVD name
emulator -avd Pixel_10_Pro_XL # boot it (use your name)

# in another terminal, confirm it's ready:
adb devices                  # emulator-5554    device
adb shell getprop sys.boot_completed # → 1 when fully booted
```

First boot takes 1–2 min. Leave it running for the rest of the session.

Appium Inspector — live demo

1. Start the server: `appium` (**4723**).
2. Open **Appium Inspector** → **Remote Path**
`/`, server `127.0.0.1`, port `4723`.
3. Paste these capabilities → **Start Session**.
4. Click an element → read its **accessibility id**
/ id / xpath on the right.

```
{  
  "platformName": "Android",  
  "appium:automationName": "UiAutomator2",  
  "appium:appPackage": "com.android.settings",  
  "appium:appActivity": ".Settings",  
  "appium:newCommandTimeout": 120  
}
```

This is how you'll find the **locators** your scripts use. Watch what I click.

Two stacks, one protocol — what's the difference?

● Node — WebdriverIO + Mocha

- **WebdriverIO** = a full test runner / framework
- **Mocha** = test structure (`describe` / `it`)
- Runner **auto-creates + quits** the session (`wdio.conf.js`)
- **Auto-waiting** built in (`$` , `expect`)

Python — Appium-Python-Client + pytest

- **Appium-Python-Client** = a WebDriver client library
- **pytest** = test structure (functions + fixtures)
- **You** create + `quit()` the session (in a fixture)
- **No auto-wait** — add `WebDriverWait` / `implicitly_wait`

Key difference: WDIO is a *framework* that manages session + waits **for you**; the Python stack is a *library* where **you** drive them. Same WebDriver underneath.

Lab 1: Your First Script

1:30 → 2:00

Full instructions: [first-script/README.md](#)

What our first script does

Tiny on purpose — a **guaranteed green** first win:

1. **Connect** to `http://127.0.0.1:4723` with Android capabilities
2. **Launch** the app
3. **Find** one element + **tap / read** it
4. **Assert** something is true
5. **Quit** the session

Same five steps in every Appium test you'll ever write. Today: just see them work.

The script — Node (WebdriverIO + Mocha)

```
// wdio.conf.js – capabilities + connect + quit live here
export const config = {
  hostname: '127.0.0.1', port: 4723,
  specs: ['./test/specs/**/*.e2e.js'],
  framework: 'mocha',
  capabilities: [{
    platformName: 'Android',
    'appium:automationName': 'UiAutomator2',
    'appium:appPackage': 'com.android.settings',
    'appium:appActivity': '.Settings',
  }],
}
```

```
// test/specs/first.e2e.js – your spec just does find + assert
describe('Android Settings', () => {
  it('opens and shows the Settings home screen', async () => {
    // resource-id is stable across Android versions & languages
    const home = $('//*[@resource-id="com.android.settings:id/homepage_container"]')
    await expect(home).toBeDisplayed() // ✅ auto-waits
  })
})
```

Running the Node test

Each step in its **own terminal** — leave the first two running:

```
# 1 · boot an emulator
emulator -avd Pixel_10_Pro_XL
adb devices                # → emulator-5554  device

# 2 · start the Appium server (port 4723)
appium

# 3 · run the test
cd first-script/node
npm install                # first time only
npm test                  # → wdio run ./wdio.conf.js
```

The script — Python (pytest)

```
from appium import webdriver
from appium.options.android import UiAutomator2Options
from appium.webdriver.common.appiumby import AppiumBy

def test_settings_opens():
    opts = UiAutomator2Options().load_capabilities({
        "platformName": "Android",
        "appium:automationName": "UiAutomator2",
        "appium:appPackage": "com.android.settings",
        "appium:appActivity": ".Settings",
    })
    driver = webdriver.Remote("http://127.0.0.1:4723", options=opts)
    try:
        el = driver.find_element(AppiumBy.ID, "com.android.settings:id/homepage_container")
        assert el.is_displayed() # ✅
    finally:
        driver.quit()
```

▶ Running the Python test

Terminals 1 & 2 (both OSes): `emulator -avd Pixel_10_Pro_XL` → `adb devices`, then `appium` (port 4723). **Terminal 3 — run the test:**

🍏 macOS / Linux

```
cd first-script/python
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pytest
```

🪟 Windows (PowerShell)

```
cd first-script\python
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pytest
```

Why a venv? Homebrew/system Python (PEP 668) blocks global `pip install` — the venv sidesteps it. PowerShell blocks activation? `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once. (cmd.exe: `.venv\Scripts\activate.bat .`)



Read it line by line

Both scripts are the **same five steps**:

Step	Node (WebdriverIO + Mocha)	Python (pytest)
Capabilities	<code>capabilities: [{...}]</code> in <code>wdio.conf.js</code>	<code>UiAutomator2Options().load_capabilities(...)</code>
Connect	<i>runner auto-starts the session</i>	<code>webdriver.Remote(url, options)</code>
Find	<code>\$('...')</code>	<code>driver.find_element(...)</code>
Assert	<code>expect(el).toBeDisplayed()</code>	<code>assert ...</code>
Quit	<i>runner auto-ends the session</i>	<code>driver.quit()</code>

Different syntax, **identical concepts** — because both speak WebDriver.
With the WebdriverIO runner, **connect + quit are handled for you**.

Run it (with **appium** already running)

Node

```
cd first-script/node  
npm install  
npm test
```

Python

```
cd first-script/python  
pip install -r requirements.txt  
pytest
```

Watch the **emulator**: Settings opens, the session closes, your terminal goes **green**. 🎉
That's a passing UI test.

If it goes red — quick triage

- **ECONNREFUSED** / can't connect → is `appium` running in another terminal?
- **No device** / session won't start → `adb devices` shows your emulator as `device` ?
- **appium: command not found** → reopen terminal; reinstall `npm i -g appium`.
- **driver not found** → `appium driver install uiautomator2`.

Full table in [first-script/README.md](#) + prerequisites Troubleshooting.

Break — 10 min

2:00 → 2:10

We're 2 hours in. Stretch, grab water — **leave your emulator + `appium` running** for Part 2.

Part 2 — Locators, Commands & a Real Flow

2:10 → 4:00

Now we make tests that actually *do* something.

Recap: capabilities start the session...

```
{ "platformName": "Android", "appium:automationName": "UiAutomator2",  
  "appium:appPackage": "...", "appium:appActivity": "..." }
```

...but a session is useless until you can **point at elements** and **act on them**. That's Part 2:

1. **Locators** — *how do I find an element?*
2. **Commands** — tap, type, read text, wait
3. **Lab 2** — put it together: a **login flow**



Locator strategies — strategy + value

You hand the driver a **strategy + value**; it returns the matching element. Some strategies work on **both** platforms; each OS also has its own **native** strategy.

	Strategies
Cross-platform	accessibility id · id · class name · xpath
 Android-native	UiAutomator (<code>UiSelector</code>)
 iOS-native	-ios predicate string · -ios class chain

You read every one of these straight off the right-hand panel in **Appium Inspector**.



Android locator strategies

Strategy	Matches	Example value
accessibility id	content-desc	login-button
id	the resource-id	com.example:id/username
class name	the widget type	android.widget.Button
xpath	a path in the tree	//android.widget.EditText[@text="Email"]
UiAutomator	a UiSelector query	see below

```
// UiSelector – Android's most powerful, fastest native query
new UiSelector().text("Login")
new UiSelector().resourceId("com.example:id/username")
new UiSelector().description("login-button") // content-desc
new UiSelector().className("android.widget.Button").textContains("Log")
new UiSelector().resourceId("...:id/row").childSelector(new UiSelector().text("OK"))
new UiScrollable(new UiSelector().scrollable(true)).scrollIntoView(new UiSelector().text("Bottom"))
```

Node: `$( 'android=new UiSelector().text("Login")' )` · Python: `AppiumBy.ANDROID_UIAUTOMATOR`



iOS locator strategies

Strategy	Matches	Example value
accessibility id	<code>accessibilityIdentifier</code> / <code>name</code>	<code>login-button</code>
class name	the element type	<code>XCUIElementTypeButton</code>
-ios predicate string	an NSPredicate query	<i>see below</i>
-ios class chain	a fast, path-like query	<i>see below</i>
xpath	a path in the tree	<code>//XCUIElementTypeButton[@name="login"]</code>

predicate string

```
type == 'XCUIElementTypeButton' AND name == 'login'
label BEGINSWITH 'User'
value CONTAINS 'foo'
visible == 1
```

class chain

```
**/XCUIElementTypeButton[`name == "login"`]
**/XCUIElementTypeCell[2]
**/XCUIElementTypeTextField[`label == "Email"`]
```

Node: `$('-ios predicate string:...') / $('-ios class chain:...')` · Python: `AppiumBy.IOS_PREDICATE / IOS_CLASS_CHAIN` · Both are **faster + sturdier than xpath** on iOS.

xpath, in depth

<code>//*[@text='Login']</code>	descendant + attribute match
<code>/hierarchy/.../android.widget.Button</code>	absolute · <code>//</code> = any depth, <code>/</code> = direct child
<code>//android.widget.Button[@text='OK' and @enabled='true']</code>	multiple attributes
<code>//*[contains(@text, 'Log')]</code>	<code>contains()</code> · <code>starts-with(@text, 'Sign')</code>
<code>(//android.widget.EditText)[1]</code>	index – the 1st match (1-based)

Axes — navigate *relative* to an element you found:

<code>//*[@text='Email']/..</code>	parent
<code>//*[@text='Email']/following-sibling::android.widget.EditText</code>	next sibling
<code>//*[@text='Submit']/preceding-sibling::*</code>	earlier sibling
<code>//*[@text='OK']/ancestor::android.widget.LinearLayout</code>	any ancestor

 Android uses `android.widget.*` classes;  iOS uses `XCUIElementType*`.

xpath is the **slowest + most brittle** strategy (it walks the whole tree) — prefer it last.



Strategy → selector, in code

Strategy	WebdriverIO selector	Python AppiumBy
accessibility id	<code>\$('~login-button')</code>	<code>ACCESSIBILITY_ID</code>
id	<code>\$('id=...:id/username')</code>	<code>ID</code>
class name	<code>\$('android.widget.Button')</code>	<code>CLASS_NAME</code>
xpath	<code>\$('//...')</code>	<code>XPATH</code>
Android UiAutomator	<code>\$('android=new UiSelector()...')</code>	<code>ANDROID_UIAUTOMATOR</code>
iOS predicate string	<code>\$('-ios predicate string:...')</code>	<code>IOS_PREDICATE</code>
iOS class chain	<code>\$('-ios class chain:...')</code>	<code>IOS_CLASS_CHAIN</code>

`~name` is WebdriverIO shorthand for **accessibility id** — the one you'll reach for most.

Which locator should I prefer?

1. **accessibility id**  — stable, **cross-platform**, set by devs on purpose.
2. **Android: id / UiSelector** before xpath. **iOS: predicate string / class chain** before xpath.
3. **xpath**  — flexible but **slow + brittle** (worst on iOS); breaks when the tree shifts. Last resort.
4. **class name** alone is too broad (many `Button` s) — combine it or avoid.

 Rule of thumb: **the more a locator depends on layout, the more it breaks.**

Ask your devs to add `content-desc` / `accessibilityIdentifier` test ids — it helps everyone.

Core commands — the verbs of a test

Node (WebdriverIO)

```
const el = $('~username')
await el.setValue('alice') // sendKeys (clears first)
await el.addValue('!')    // append
await $('~login').click() // tap
const t = await $('~msg').getText()
await $('~box').clearValue()
```

Python (Appium-Python-Client)

```
el = driver.find_element(AppiumBy.ACCESSIBILITY_ID, "username")
el.send_keys("alice") # type
driver.find_element(AppiumBy.ACCESSIBILITY_ID, "login").click() # tap
t = driver.find_element(AppiumBy.ACCESSIBILITY_ID, "msg").text
el.clear()
```

Find → **act** (tap / type) → **read** (`getText`) → assert. That's every test.

Waits — never use `sleep`

A hard `sleep(3)` is **slow when it works and flaky when it doesn't**. Wait for a *condition*.

Implicit — one global "retry-find for up to N seconds"

```
driver.implicitly_wait(10) # Python
```

WebdriverIO auto-waits for you

```
// $ commands auto-retry; assertions wait too  
await $('~home').waitForDisplayed()  
await expect($('~home')).toBeDisplayed()
```

Explicit — wait for a *specific* condition

```
WebDriverWait(driver, 15).until(  
    EC.visibility_of_element_located(  
        (AppiumBy.ACCESSIBILITY_ID, "home")))
```

Prefer **explicit / condition-based** waits. They make tests fast *and* reliable.

Lab 2 — Automate a Login Flow

3:15 → 3:55

Full instructions: [login-lab/README.md](#)

Our app: the **taqelah demo-app** 🛒

A small **Flutter** e-commerce app — github.com/taqelah/demo-app.

1. caps launch the app (`com.taqelah.demo_app / .MainActivity`)
2. FIND username → tap → sendKeys `"emma@demoapp.com"`
3. FIND password → tap → sendKeys `"10203040"`
4. FIND "Login" → tap
5. ASSERT "View All" is displayed on the home screen ✓

⚠ It's **Flutter** → the view tree has no **resource-id**s. Perfect: we use what the lecture taught — **UiAutomator class+instance** for the fields, **accessibility id** for the buttons.

Step 1 — the locators (confirm in Inspector)

Install the app (`adb install DemoApp-v1.0.0.apk`), open **Appium Inspector**, **Start Session** with the caps above, and click each element. Note: **no resource-id** — so:

Element	Strategy	Value
Username field	UiAutomator	<code>...className("android.widget.EditText").instance(0)</code>
Password field	UiAutomator	<code>...className("android.widget.EditText").instance(1)</code>
Login button	accessibility id	Login
Post-login element	accessibility id	View All <i>(assert, don't tap)</i>

Two fields, same class → tell them apart with `.instance(0)` / `.instance(1)`.

Step 2 — the script (Node · WebdriverIO + Mocha)

```
const UI = 'android=new UiSelector().className("android.widget.EditText")'
describe('Login flow', () => {
  it('logs in with valid credentials', async () => {
    await `${UI}.instance(0)`).click()
    await `${UI}.instance(0)`).setValue('emma@demoapp.com')
    await `${UI}.instance(1)`).click()
    await `${UI}.instance(1)`).setValue('10203040')
    await $('~Login').click()
    await expect($('~View All')).toBeDisplayed() // ✅ assert, don't tap
  })
})
```

Step 2 — the script (Python · pytest)

```
UA = AppiumBy.ANDROID_UIAUTOMATOR
FIELD = 'new UiSelector().className("android.widget.EditText").instance({})'
def test_login_succeeds(driver):
    wait = WebDriverWait(driver, 15)
    user = wait.until(EC.presence_of_element_located((UA, FIELD.format(0))))
    user.click(); user.send_keys("emma@demoapp.com")
    pwd = driver.find_element(UA, FIELD.format(1))
    pwd.click(); pwd.send_keys("10203040")
    driver.find_element(AppiumBy.ACCESSIBILITY_ID, "Login").click()
    assert wait.until(EC.visibility_of_element_located( # assert, don't tap
        (AppiumBy.ACCESSIBILITY_ID, "View All"))).is_displayed()
```

▶ Running login-lab — Node

One-time: install the demo app → `adb install DemoApp-v1.0.0.apk` (from the GitHub release; see Step 1). Terminals 1 & 2: `emulator -avd Pixel_10_Pro_XL` + `adb devices`, then `appium` (port 4723).

```
# Terminal 3 – run the test (npm is identical on macOS & Windows)
cd login-lab/node                # Windows: cd login-lab\node
npm install                      # first time only
npm test                         # → wdio run ./wdio.conf.js
```

Logs in as `emma@demoapp.com`, taps **Login**, asserts **View All** is visible. ✓

▶ Running login-lab — Python

One-time: `adb install DemoApp-v1.0.0.apk` (Step 1). Terminals 1 & 2: `emulator -avd Pixel_10_Pro_XL` + `adb devices`, then `appium`. **Terminal 3 — run the test:**

macOS / Linux

```
cd login-lab/python
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pytest
```

Windows (PowerShell)

```
cd login-lab\python
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pytest
```

Why a venv? Homebrew/system Python (PEP 668) blocks global `pip install` — the venv sidesteps it. PowerShell blocks activation? `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once. (cmd.exe: `.venv\Scripts\activate.bat`.)

Run it & watch

- The emulator **types** `emma@demoapp.com` + password, taps **Login**, and the home screen appears.
- The test **asserts** the **View All** button is visible. Your terminal goes **green**:

```
✓ Login flow logs in with valid credentials    (Node)
==== 1 passed ====                             (Python)
```

 A real, **multi-step** UI test: find → tap → type → tap → assert — against a real Flutter app.

Wrap-up

3:55 → 4:00

What you did today

-  Mapped the landscape: **iOS vs Android, native / hybrid / web**
-  Understood **why Appium** and **how it's wired** (client → server → driver → device)
-  Verified your environment and used **Appium Inspector**
-  Ran a **passing Appium test** — in *two* languages
-  Learned **locator strategies + core commands** (tap, sendKeys, getText, waits)
-  Automated a real **login flow** end-to-end

You now have a working setup *and* can write a real multi-step test. That's the hard part most people quit on.

Homework before Session 2

1. Get **both labs green** on your own machine if we ran short.
2. In the login lab, **swap each locator** (try accessibility id, then id, then xpath) and feel which is most stable.

Stuck? Post in the cohort channel with your OS, the command, and the full error.

Next: Session 2 — Real, maintainable tests

- **Page Objects** — stop repeating locators; organize tests for scale
- **Sync & waits** patterns that kill flakiness for good
- **Gestures** — swipe, scroll, long-press — and **data-driven** tests
- ...then from **Session 3** we start letting **Claude** write these for us 

Questions?

 contact@taqelah.sg · cohort channel

See you next Sunday.